

Table 6-3: How Power Sets Grow as New Elements (in Bold) Are Added to the Set

Set with new element	New sets to the power set	Complete power set
{ }	{ }	{{ }}
{A}	{A}	{{ }, { A }
{A, B}	{B}, {A, B}	{{ }, {A}, { B }, { A, B }
{A, B, C}	{C}, {A, C}, {B, C}, {A, B, C}	{{ }, {A}, {B}, { C }, {A, B}, { A, C }, { B, C }, { A, B, C }
{A, B, C, D}	{D}, {A, D}, {B, D}, {C, D}, {A, B, D}, {A, C, D}, {B, C, D}, {A, B, C, D}	{{ }, {A}, {B}, {C}, { D }, {A, B}, {A, C}, { A, D }, {B, C}, { B, D }, { C, D }, {A, B, C}, {A, B, D}, {A, C, D}, { B, C, D }, { A, B, C, D }

The power sets of larger sets are similar to the power sets of smaller sets, hinting that we can create a recursive function to generate them. The base case is an empty set, and its power set is a set of just the empty set. We can use a head-tail technique for this recursive function. For each new element we add, we want to get the power set of the tail to add to our full power set. We also add the head element to each set in the tail power set. Together, these form the full power set for the `chars` argument.

Let's ask the three recursive algorithm questions about our power set algorithm:

What is the base case? If `chars` is the blank string (the empty set), the function returns an array with just a blank string, since the empty set is the only subset of the empty set.

What argument is passed to the recursive function call? The tail of `chars` is passed.

How does this argument become closer to the base case? Since the recursive calls remove the heads from the `chars` arguments, eventually the `chars` argument becomes the blank string.

The `getPowerSet()` recursive function is implemented in *powerSet.py*:

Python

```
def getPowerSet(chars, indent=0):
    debugMsg = '.' * indent + 'In getPowerSet("' + chars + '")'
    print(debugMsg + ', start.')
```